RedHat OpenShift - Développement niveau 1 - Applications de conteneurisation

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Rappel Jour 2 et Programme Jour 3

Jour 2 — Acquis :

- Déploiement avec `oc new-app`, Templates, S2I
- Gestion des images, registre interne
- Microservices, PVC, Autoscaling

Jour 3 — Programme :

Module	Thème	Durée
7	Supervision et maintenance d'OpenShift	2h
8	Migration des applications	2h
9	Etude de cas final	2h

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Module 7

Supervision et Maintenance d'OpenShift

Durée : 2 heures

Stack de monitoring OpenShift

OpenShift intègre une pile de surveillance complète :

```
┌──────────────────────────────────────────┐
│              Console OpenShift           │
│           (Dashboards intégrés)          │
├──────────────────────────────────────────┤
│   Grafana    │      Alertmanager         │
│  (Tableaux)  │  (Gestion des alertes)    │
├──────────────────────────────────────────┤
│              Prometheus                  │
│     (Collecte et stockage métriques)     │
├──────────────────────────────────────────┤
│    node-exporter │ kube-state-metrics    │
│     (Métriques système et K8s)           │
└──────────────────────────────────────────┘
```

Métriques avec Prometheus

```
# Accéder à Prometheus via la console
# Monitoring > Metrics (perspective Admin)

# Requêtes PromQL utiles

# CPU utilisé par namespace
sum(rate(container_cpu_usage_seconds_total{
  namespace="mon-projet"
}[5m])) by (pod)

# Mémoire utilisée par pod
container_memory_working_set_bytes{
  namespace="mon-projet"
} / 1024 / 1024

# Nombre de pods par état
kube_pod_status_phase{namespace="mon-projet"}

# Taux d'erreurs HTTP
sum(rate(http_requests_total{
```

Monitoring applicatif personnalisé

```yaml
# ServiceMonitor pour scraper les métriques de votre app
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: mon-app-monitor
  labels:
    app: mon-app
spec:
  selector:
    matchLabels:
      app: mon-app
  endpoints:
    - port: http
      path: /metrics
      interval: 30s
  namespaceSelector:
```

Gestion des logs

```
# Logs d'un pod
oc logs mon-pod

# Logs en temps réel
oc logs -f mon-pod

# Logs d'un conteneur spécifique dans un pod multi-conteneurs
oc logs mon-pod -c mon-conteneur

# Logs d'un build
oc logs -f bc/mon-app

# Logs des précédentes instances (après un crash)
oc logs mon-pod --previous

# Logs avec filtre temporel
oc logs mon-pod --since=1h
oc logs mon-pod --since-time="2024-01-15T10:00:00Z"
```

EFK/Loki Stack pour les logs centralisés

OpenShift peut déployer un système de logs centralisé :

Composant	Rôle
Fluentd / Vector	Collecte des logs sur chaque nœud
Elasticsearch / Loki	Stockage et indexation
Kibana / Grafana	Visualisation et recherche

```
# Installer l'opérateur de logging
# Via OperatorHub > Red Hat OpenShift Logging

# Créer une instance ClusterLogging
oc apply -f - <<EOF
```

Alertes et notifications

```yaml
# PrometheusRule pour des alertes personnalisées
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: mon-app-alerts
  namespace: mon-projet
spec:
  groups:
    - name: mon-app.rules
      rules:
        - alert: HighErrorRate
          expr: |
            sum(rate(http_requests_total{
              status=~"5..",app="mon-app"
            }[5m])) > 0.1
          for: 5m
          labels:
            severity: critical
          annotations:
            summary: "Taux d'erreur élevé sur {{ $labels.app }}"
            description: "Plus de 10% d'erreurs 5xx depuis 5 minutes"
        - alert: PodCrashLooping
          expr: rate(kube_pod_container_status_restarts_total{namespace="mon-projet"}[15m]) > 0
          for: 5m
```

Diagnostic et troubleshooting

```
# Vérifier l'état global
oc get nodes
oc get clusteroperators
oc adm top nodes
oc adm top pods -n mon-projet

# Diagnostic d'un pod en erreur
oc get pods -n mon-projet
oc describe pod <pod-name>
oc logs <pod-name>
oc get events -n mon-projet --sort-by='.lastTimestamp'

# Debug interactif
oc debug pod/<pod-name>
oc rsh <pod-name>
oc exec <pod-name> -- <commande>

# Collecter des diagnostics
```

Problèmes courants et solutions

Symptôme	Commande de diagnostic	Cause probable
Pod en `Pending`	`oc describe pod`	Ressources insuffisantes, PVC manquant
Pod en `CrashLoopBackOff`	`oc logs --previous`	Erreur applicative, config manquante
Pod en `ImagePullBackOff`	`oc describe pod`	Image introuvable, auth registre
Pod en `ErrImagePull`	`oc get events`	Registre inaccessible
Service inaccessible	`oc get endpoints`	Selector ne matche pas les labels
Route ne fonctionne pas	`oc describe route`	Certificat TLS, service backend

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Gestion des ressources du cluster

```
# Voir la capacité du cluster
oc adm top nodes
oc describe node <node-name>

# Quotas par projet
oc get resourcequota -n mon-projet
oc describe resourcequota -n mon-projet

# Drain d'un nœud pour maintenance
oc adm cordon <node-name>    # Empêcher les nouveaux pods
oc adm drain <node-name> \
  --ignore-daemonsets \
  --delete-emptydir-data      # Évacuer les pods
oc adm uncordon <node-name>   # Remettre en service

# Mise à jour du cluster
```

Module 8

Migration des Applications vers OpenShift

Durée : 2 heures

Stratégies de migration

Stratégie	Description	Complexité
Lift & Shift	Conteneuriser tel quel	Faible
Replatform	Adapter pour la plateforme	Moyenne
Refactor	Rearchitecturer en microservices	Élevée
Rebuild	Réécrire pour le cloud natif	Très élevée

> **Recommandation** : commencer par Lift & Shift, puis itérer vers Refactor.

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Analyse de l'application à migrer

Checklist de migration :

- [ ] L'application écoute sur un port configurable (> 1024) ?
- [ ] L'application peut s'exécuter en tant que non-root ?
- [ ] Les données persistantes sont séparées du code ?
- [ ] La configuration est externalisable (env vars, fichiers) ?
- [ ] L'application est stateless (ou le state est dans un service externe) ?
- [ ] Les dépendances système sont identifiées ?
- [ ] Les health checks sont implémentés ?

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Étape 1 : Conteneuriser l'application

Application Java traditionnelle :

```
FROM registry.access.redhat.com/ubi8/openjdk-17:latest

WORKDIR /opt/app

# Copier le JAR
COPY target/mon-app.jar app.jar

# Variables d'environnement pour la config
ENV JAVA_OPTS="-Xmx512m -Xms256m"
ENV SERVER_PORT=8080

# Port non-root
EXPOSE 8080

# Utilisateur non-root
```

Étape 2 : Externaliser la configuration

Avant (hardcodé) :

```
# application.properties dans le JAR
db.host=192.168.1.100
db.port=5432
db.name=production
```

Après (externalisé) :

```
# ConfigMap OpenShift
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
```

Étape 3 : Gérer la persistance

```
# Créer un PVC pour les données de l'application
oc apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
EOF

# Monter dans le déploiement
oc set volume deployment/mon-app \
  --add --name=data-volume \
  --type=persistentVolumeClaim \
```

Étape 4 : Ajouter les health checks

```
# Ajouter une liveness probe
oc set probe deployment/mon-app \
  --liveness \
  --get-url=http://:8080/actuator/health \
  --initial-delay-seconds=60 \
  --period-seconds=10

# Ajouter une readiness probe
oc set probe deployment/mon-app \
  --readiness \
  --get-url=http://:8080/actuator/health/readiness \
  --initial-delay-seconds=30 \
  --period-seconds=5

# Vérifier
```

Étape 5 : Exposer et tester

```
# Créer le service (si pas déjà fait)
oc expose deployment/mon-app --port=8080

# Créer la route
oc expose svc/mon-app
oc get route mon-app

# Tester l'application
curl http://mon-app-mon-projet.apps-crc.testing/

# Tester les health checks
curl http://mon-app-mon-projet.apps-crc.testing/actuator/health

# Vérifier les métriques
oc adm top pod
```

Migration d'une base de données

```
# Déployer PostgreSQL sur OpenShift
oc new-app postgresql:13-el8 \
  --name=postgresql \
  -e POSTGRESQL_USER=app_user \
  -e POSTGRESQL_PASSWORD=secret \
  -e POSTGRESQL_DATABASE=app_db

# Ajouter un volume persistant
oc set volume deployment/postgresql \
  --add --name=pg-data \
  --type=persistentVolumeClaim \
  --claim-name=postgresql-data \
  --mount-path=/var/lib/pgsql/data \
  --claim-size=20Gi

# Importer les données
oc rsh postgresql-xxxxx
psql -U app_user -d app_db < /tmp/dump.sql

# Ou depuis l'extérieur via port-forward
```

Migration Toolkit for Applications (MTA)

MTA analyse et aide à la migration :

```
# Installer l'opérateur MTA via OperatorHub
# 1. Rechercher "Migration Toolkit for Applications"
# 2. Installer l'opérateur
# 3. Créer une instance Tackle

# MTA fournit :
# - Analyse des dépendances
# - Estimation de l'effort de migration
# - Recommandations de modification
# - Rapports détaillés
```

Module 9

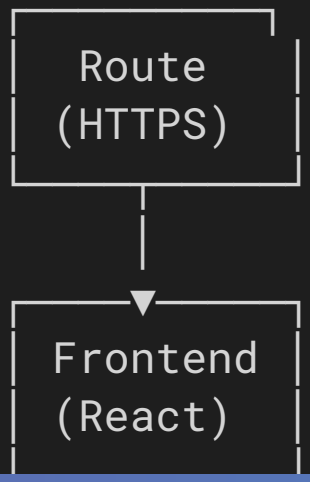
Etude de Cas Final

Durée : 2 heures

Projet final — E-Commerce Microservices

Contexte : Migrer et déployer une application e-commerce sur OpenShift

Architecture cible :



Projet final — Livrables attendus

Les participants doivent :

1. **Créer le projet** et configurer les rôles RBAC
2. **Déployer la base de données** PostgreSQL avec PVC
3. **Construire** les images des microservices (S2I ou Dockerfile)
4. **Publier** les images dans le registre interne
5. **Configurer** les ConfigMaps et Secrets
6. **Déployer** chaque microservice
7. **Exposer** l'application via une Route HTTPS

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Récapitulatif de la formation

Jour 1 : Fondamentaux

- Conteneurisation, architecture OpenShift
- Installation, CLI, Console Web
- Projets, RBAC, SCC

Jour 2 : Développement et déploiement

- Templates, S2I, BuildConfig
- Images, registre interne
- Microservices, PVC, Autoscaling

Utopios® Tous droits réservés - lot 4-002 20/03/2026

Récapitulatif de la formation

Jour 3 : Administration et migration

- Monitoring (Prometheus/Grafana)
- Troubleshooting, logs
- Stratégies de migration
- Etude de cas complète

Pour aller plus loin

Certifications Red Hat :

- **EX180** : Red Hat Certified Specialist in Containers and Kubernetes
- **EX280** : Red Hat Certified Specialist in OpenShift Administration
- **DO288** : Red Hat OpenShift Development

Ressources :

- docs.openshift.com
- learn.openshift.com (labs interactifs)
- github.com/openshift
- Red Hat Developer Program (developer.redhat.com)

Utopios® Tous droits réservés - lot 4-002 20/03/2026